

add.py 2.0

Build 3D models with nothing but code -- in Python or in C++



What is new in 2.0

Boolean operations

union, intersect, difference -- written from scratch, no libraries.

Regular polyhedra and a geodesic sphere

tetrahedron ... icosahedron, centred; sphere() is now a dome of triangles.

Smooth surfaces

catmull_clark, and smooth(): the generalised Catmull-Clark algorithm, any number of cells per edge.

Vertex tools

set_vertex, neighbors, valence, dual, truncate -- a football from the vertices of an icosahedron.

Named surfaces, colour functions, parts

25 surfaces by name; color=lambda u, v: ...; beam, wheel, gear, roof, tree, text ...

Sketchfab-ready export

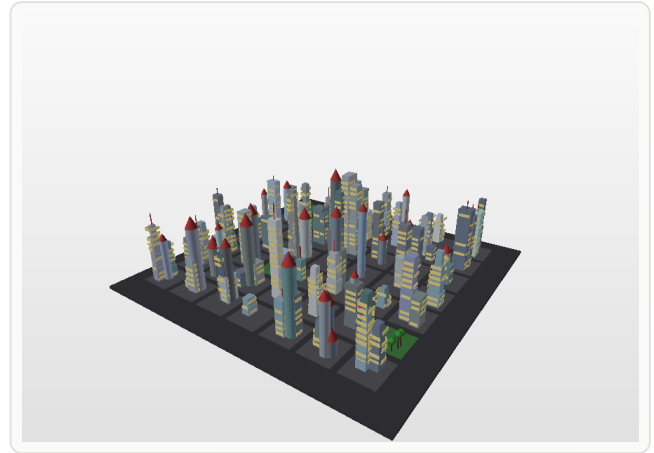
check() watches the 50 MB / 50 colour limits; glass (transparent) and image textures go into the .mtl.

Everything written for 1.2 keeps working without a single change -- and all of it now comes for C++ too: add.hpp, one header, the same functions, the same models.

REMINDER

The creative assignment

- Make a 3D model using nothing but program source code.
- File format: OFF or OBJ.
- At least 10 000 polygons.
- At least 3 different colours.
- The algorithm must contain a for or while loop.
- At least one parameter that changes the shape.
- No 3D modelling programs.
- Individually or in pairs.



One number (SEED) decides the whole city

add.check()

tells you which requirements are already met.

Recommended tools

Python 3

Nothing else to install: add.py uses only math and random.

add.py

One file next to your script. Downloaded from GitHub (project add3d).

tools/preview.py

Look at a model without any other program: writes a PNG.

GeoGebra

Handy for checking curve and surface formulas before coding them.

MeshLab

Viewing and converting OFF. No longer required: add.py writes OBJ itself.

Sketchfab.com

.obj + .mtl in one archive -- and anyone can turn the model in a browser.

BASICS

A model is two lists

```
OFF
8 6 0
-1 -1 -1
-1 -1 1
...
4 0 4 5 1 255 0 0
4 0 1 3 2 255 0 0
...
```

OFF -- the format's name

8 6 0 -- number of vertices, faces and edges

then the vertex coordinates

4 0 4 5 1 -- a quadrilateral joining vertices 0, 4, 5 and 1

255 0 0 -- the face's colour

Every drawing function in the library does the same thing: it works out some points and adds some faces.

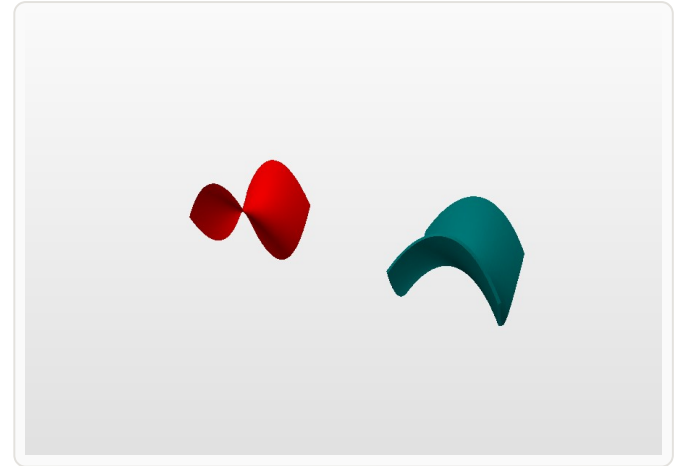
```
import add

add.box([0, 0, 0], 2, "red")
add.save("cube.off")
```

The same cube -- four lines.

Inside and outside

- Seen from outside the model, the corners of a face run counter-clockwise.
- Seen from inside -- clockwise.
- A face the wrong way round is drawn black or not at all.
- The library orients its own shapes; you only think about it when writing `add.polygon(...)` by hand.
- **`add.fix_normals(M)` fixes a whole model; `add.check()` says whether the surface is closed.**



The same saddle: sheet, two-sided, solid

The scene, layers and meshes

```
add.box([0, 0, 0], 1, "red")
brick = add.layer()      # the scene is empty
again

for i in range(10):
    add.mesh(add.move(brick, [i * 1.2, 0, 0]))

add.save("wall.off")
```

Build once -- place many times.

Transformations never change the mesh you give them -- they return a new one.

layer()

Takes the scene out as a mesh and leaves the scene empty.

mesh(M)

Puts a mesh back into the scene.

merge([...])

Joins several meshes into one.

push() / pop()

Inside a function that builds a part: put the scene aside, then bring it back.

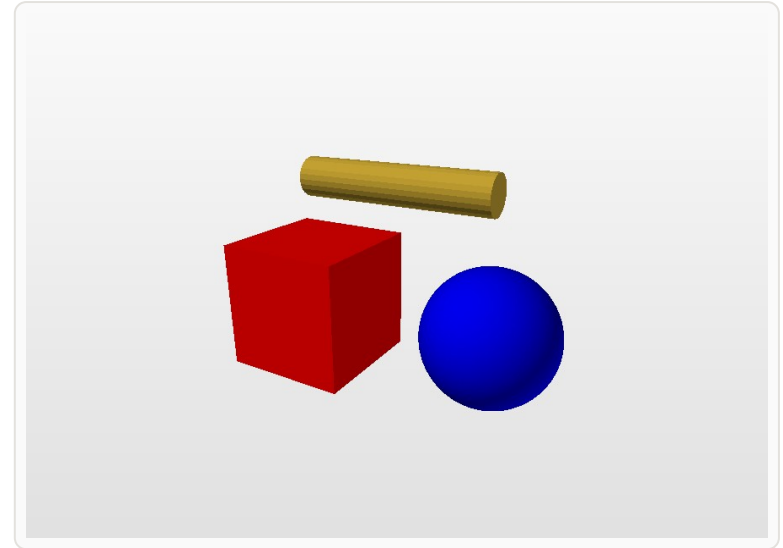
START

The first model

```
import add

add.box([0, 0, 0], 2, "red")
add.sphere([3, 0, 0], 1, 20, "blue")
add.cylinder([0, 2, 0], [3, 2, 0],
            0.3, 24, "gold")

add.check()
add.save("first.off")
```



vertices	2460	
!! polygons	2478	(need 10000)
OK colours	3	(need 3)
OK closed surface	yes	

The check() report: still short of polygons.

NEW

Only import add

```
import add

for i in range(12):
    a = 2 * add.pi * i / 12
    p = [3 * add.cos(a), 0, 3 * add.sin(a)]
    add.sphere(p, 0.4, 8, add.hsv(i / 12))

add.seed(7)
add.box([add.uniform(-3, 3), 0, 0], 1, "red")
```

add.sin, add.pi, add.sqrt ...

Everything from math, straight from add.

add.randint, add.seed,

add.Random

Everything from random, too.

A model file has exactly one import. (Plain import math still works.)

The examples in examples/ are written this way: import add and nothing else.

C++

The same model in C++

Python

```
import add

add.box([0, 0, 0], 2, "red")
add.sphere([3, 0, 0], 1, 20, "blue")
add.cylinder([0, 2, 0], [3, 2, 0],
            0.3, 24, "gold")

add.save("first.off")
```

C++

```
#include "add.hpp"

int main() {
    add::box({0, 0, 0}, 2, "red");
    add::sphere({3, 0, 0}, 1, 20, "blue");
    add::cylinder({0, 2, 0}, {3, 2, 0},
                 0.3, 24, "gold");
    add::save("first.off");
}
```

Byte for byte the same file. add.hpp is the whole library as one header, using only the C++17 standard library: every function of add.py, with the same name, parameters and defaults.

```
g++ -std=c++17 -O2 first.cpp -o first && ./first
```

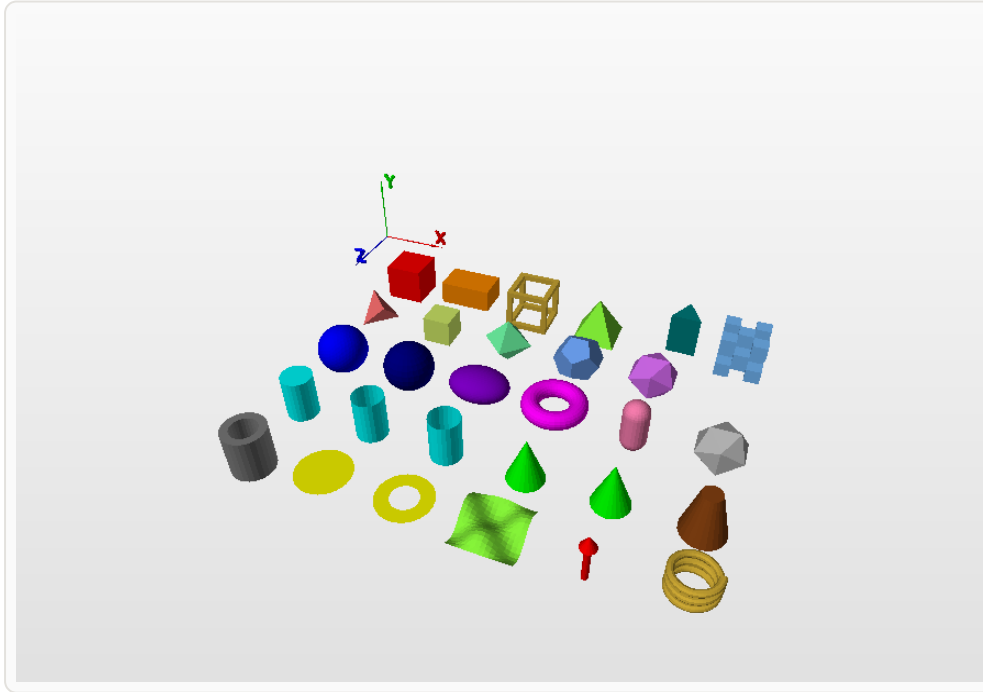
From Python to C++

Python	C++
<code>import add</code>	<code>#include "add.hpp"</code>
<code>[1, 2, 3]</code>	<code>{1, 2, 3}</code>
<code>add.bend(M, 0.4, around=2)</code>	<code>add::bend(M, 0.4, 1, 2)</code>
<code>add.bbox()</code>	<code>add::bbox()</code>
<code>lambda t: [t, 0, 0]</code>	<code>[](double t) { return add::Point{t, 0, 0}; }</code>
<code>add.make(add.box, [0, 0, 0], 2)</code>	<code>add::make([] { add::box({0, 0, 0}, 2); })</code>
<code>M.V, M.F, M.C</code>	<code>M.V, M.F, M.C</code>
<code>add.union(a, b)</code>	<code>add::union_(a, b)</code>
<code>print(add.center(M))</code>	<code>std::cout << add::center(M);</code>

No keyword arguments: give them in order (bend: angle, axis, around). A mesh left out means the scene, as in Python. union is a C++ keyword, so it is union_. Points and colours print as Python prints them.

Tested for every function, all 235 documentation examples and 43 example programs, with GCC and Clang. On ARM (a Mac with Apple Silicon) compile with -ffp-contract=off too.

Ready-made shapes



Solids

`box, cuboid, rounded_box, frame,
pyramid, prism, polyhedron, voxels`

Round

`sphere, hemisphere, uvsphere,
ellipsoid, torus, capsule, cylinder,
tube, cup, cone, cone_open, frustum,
pipe`

Flat

`polygon, triangle, quad, disc, ring,
grid`

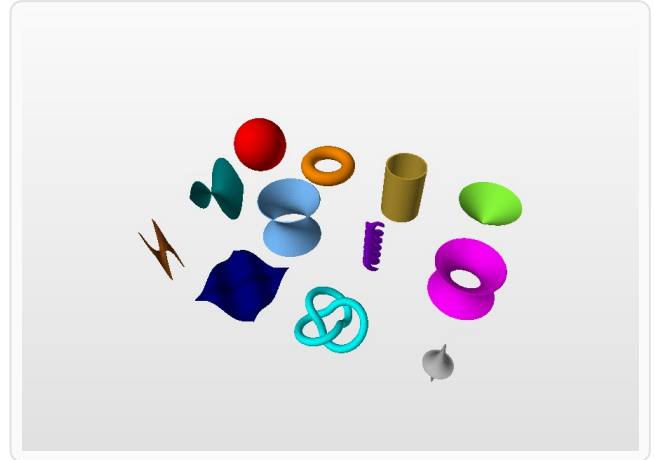
Parts

`beam, arch, stairs, gear, wheel, roof,
column, bricks, tree, pixels,
heightmap, text`

examples/02_primitives.py -- every shape in one program.

parametric() -- the central function

```
def torus(u, v):  
    return [(5 + add.cos(u)) * add.cos(v),  
            add.sin(u),  
            (5 + add.cos(u)) * add.sin(v)]  
  
add.parametric(torus, 0, 2*add.pi, 40,  
               0, 2*add.pi, 80,  
               "gold", wrap_u=True, wrap_v=True)
```



wrap_u, wrap_v

Close the surface without a seam or repeated vertices.

thickness=

Give the sheet a real thickness -- a closed solid.

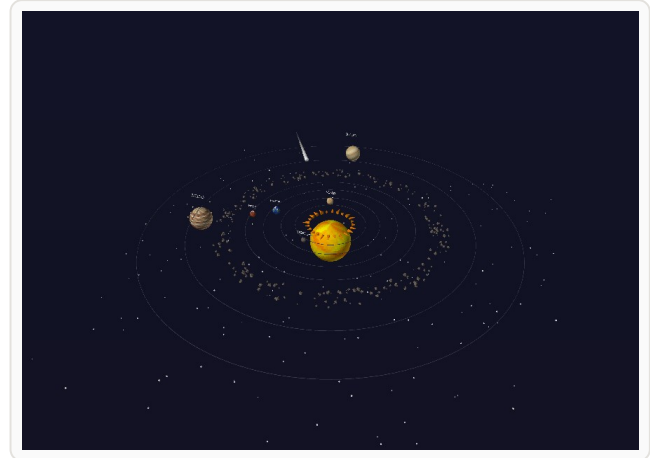
color=lambda u, v: ...

Paint every cell by its own parameters.

NEW

Colour functions

```
add.parametric(f, 0, 1, 40, 0, 1, 40,  
              color=lambda u, v: add.hsv(u))  
  
add.revolve(vase, A, B, 0, 4, 60, 40,  
           color=lambda t, a: "red" if int(t) % 2 else "white")  
  
add.curve(spiral, 0, 20, 300, 12, 0.2,  
          color=lambda t, a: add.hsv(t / 20))
```



Planets: lathes painted by (latitude, longitude)

The function is called once per cell with the surface's own two parameters: (u, v) for parametric, (t, angle) for revolve and curve, (x, z) for grid. Stripes, chequerboards, maps and gradients cost one lambda.

For a mesh you already have: color_by(M, lambda p: ...) paints by position.

One equation, twelve shapes

Gielis's superformula (2003)

$$r(t) = (|\cos(m \cdot t/4)/a|^{n_2} + |\sin(m \cdot t/4)/b|^{n_3})^{(-1/n_1)}$$

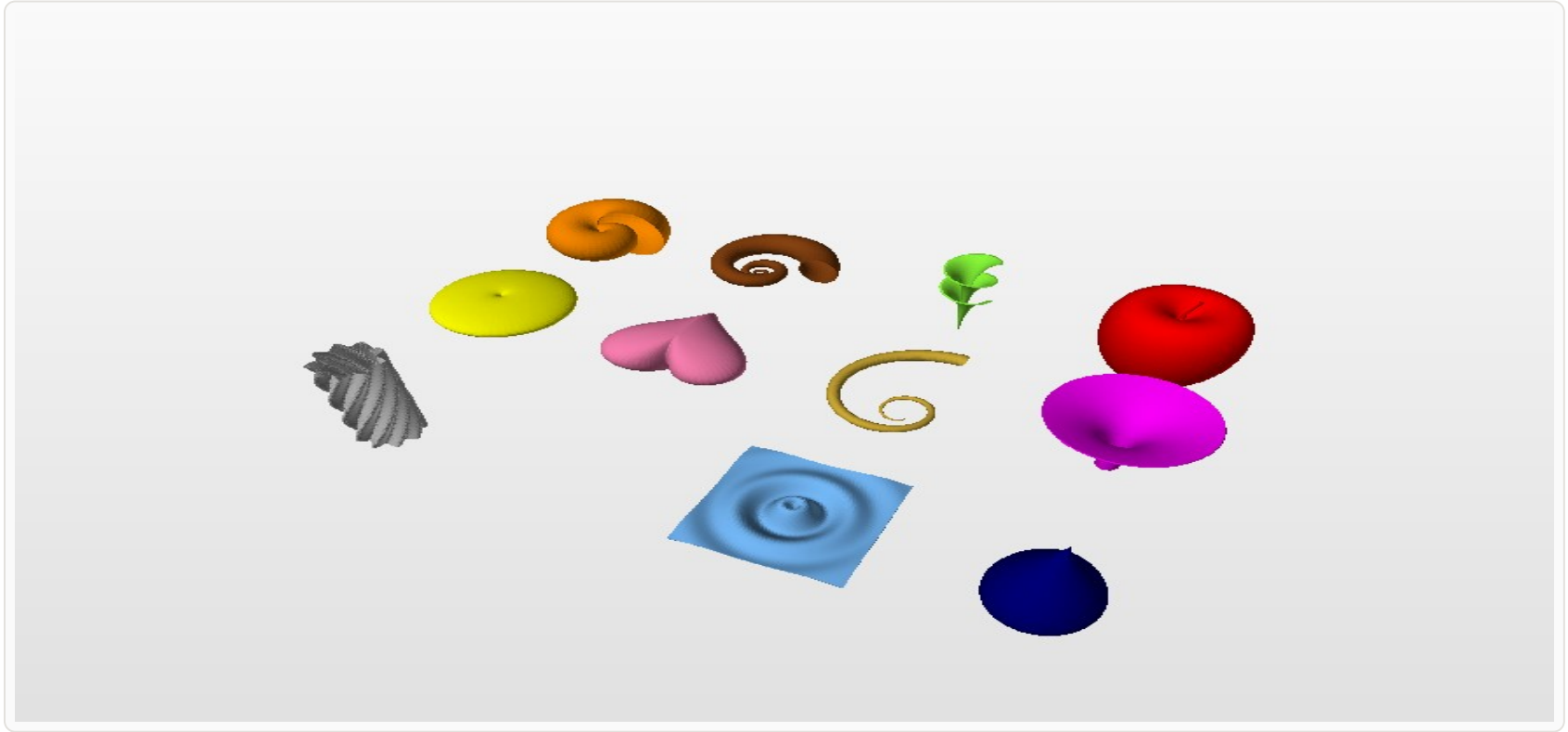
Six numbers -- and you get a starfish, a flower, a crystal or a plain ball.

Exactly what the assignment asks for: one parameter that changes the shape.

`examples/08_surfaces_superformula.py`

EXAMPLES

Surfaces that look like things



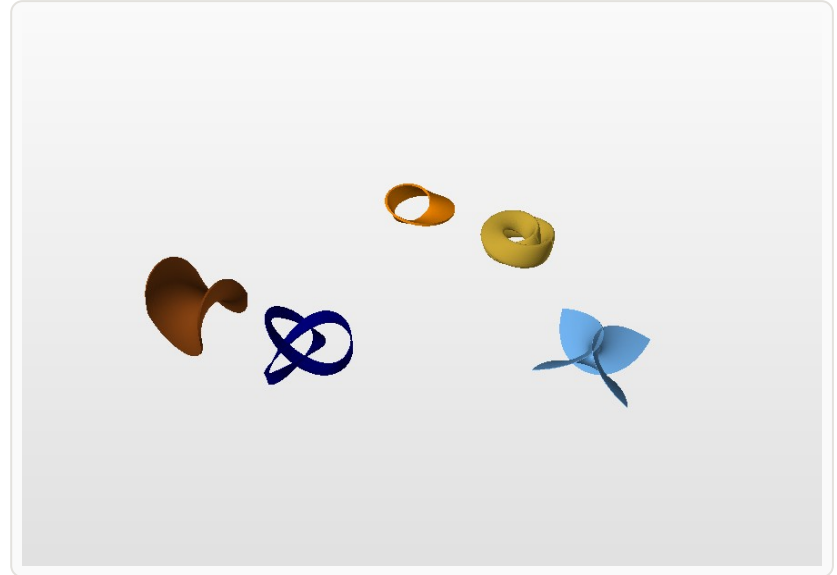
Shell, snail, Dini's surface, apple, lemon, heart, horn, trumpet, wavy column, breather, ripples, drop.
`examples/09_surfaces_nature.py`

EXAMPLES

Graphs and exotic surfaces



$y = f(x, z)$ · 10_height_fields.py



Möbius, Klein, Boy, Roman surface · 07_surfaces_exotic.py

One-sided surfaces have no "outside". Give them a thickness and they become ordinary two-sided solids.

When a surface is black from one side



A plain sheet

One good side, one "dead" side.

`double_sided=True`

Every face and its reversed copy.
Cheap; still no thickness.

`thickness=0.1`

A real solid with a wall: closed,
printable, right from every
angle.

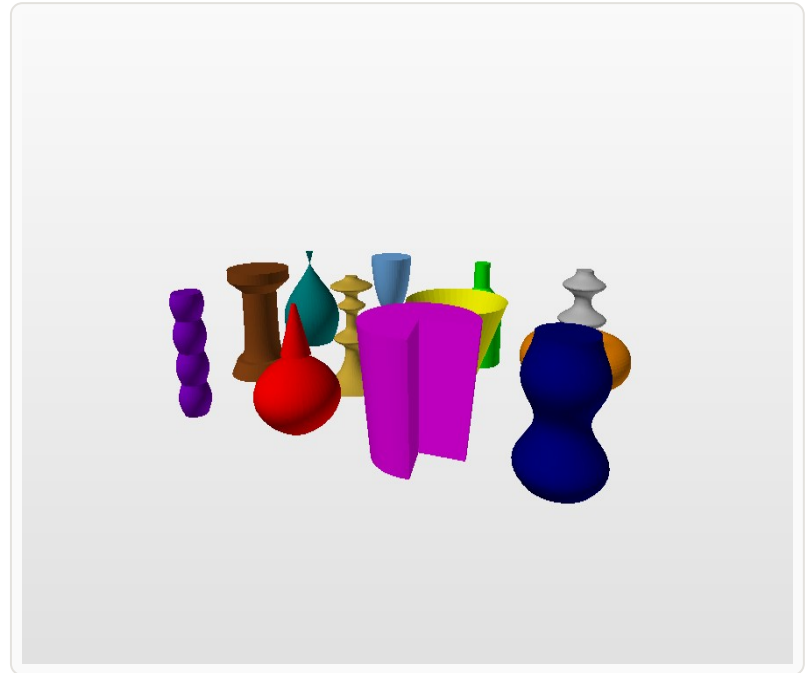
revolve() -- the lathe

```
def vase(t):  
    return [1 + 0.4*add.sin(3*t), t]  
  
add.revolve(vase, [0, 0, 0], [0, 1, 0],  
            0, 4, 60, 40, "teal")
```

profile(t) returns [radius, height]

caps=True closes the ends -- a solid ready for booleans

angle= less than a full turn cuts out a wedge

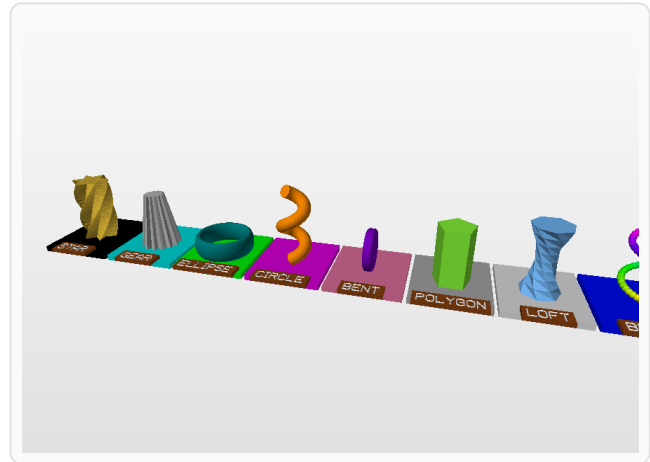


The old spin3D(A, B, S, ...) still works -- the same lathe without lids. [examples/14_lathe_gallery.py](#)

A cross-section copied, turned and stretched

```
add.extrude(star, [0, 3.4, 0], "gold",  
            steps=90, twist=1.6*add.pi,  
            scale=lambda t: 1 - 0.45*t)
```

twist and scale may be functions of the position along the path -- the shape changes as it goes. `profile_star`, `profile_gear`, `profile_rect` ... give the outlines. `examples/13_sweep_and_twist.py`, `33_cross_sections.py`



extrude

A 2D shape pulled into 3D; turned and narrowed on the way.

sweep

A cross-section pushed along any 3D path.

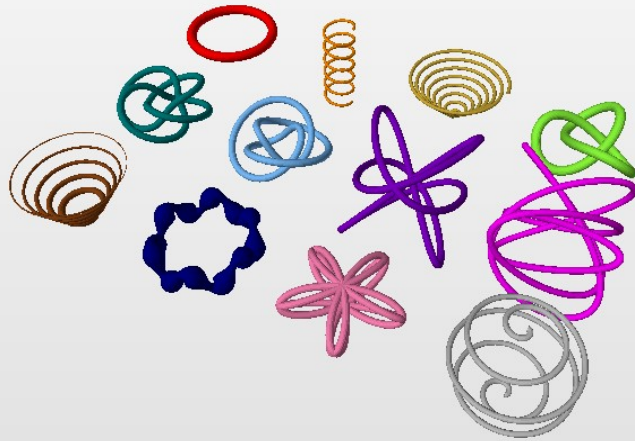
loft

A surface skinned over a sequence of cross-sections.

polyline, trace

A tube through a list of points; the path of a vector field.

curve() -- curves as tubes



```
def knot(t):  
    return [add.sin(t) +  
            2*add.sin(2*t),  
            -add.sin(3*t),  
            add.cos(t) -  
            2*add.cos(2*t)]  
  
add.curve(knot, 0, 2*add.pi,  
          300, 18, 0.28, "lime", True)
```

The radius may be a function $r(t)$ -- the tube swells and narrows. The tube does not twist on its own: a rotation-minimising frame is carried along the curve.

Transformations

move, place, align, ground

Shift; put the centre, a corner or the bottom at a point.

rotateX/Y/Z, rotate, aim

Turn about an axis -- any axis -- or aim a part along a direction.

zoom, stretch, fit

Scale: uniform, per axis, or to a size.

mirror

Reflection in a plane; the face winding is reversed too.

twist, bend, taper

Twist, bend, narrow.

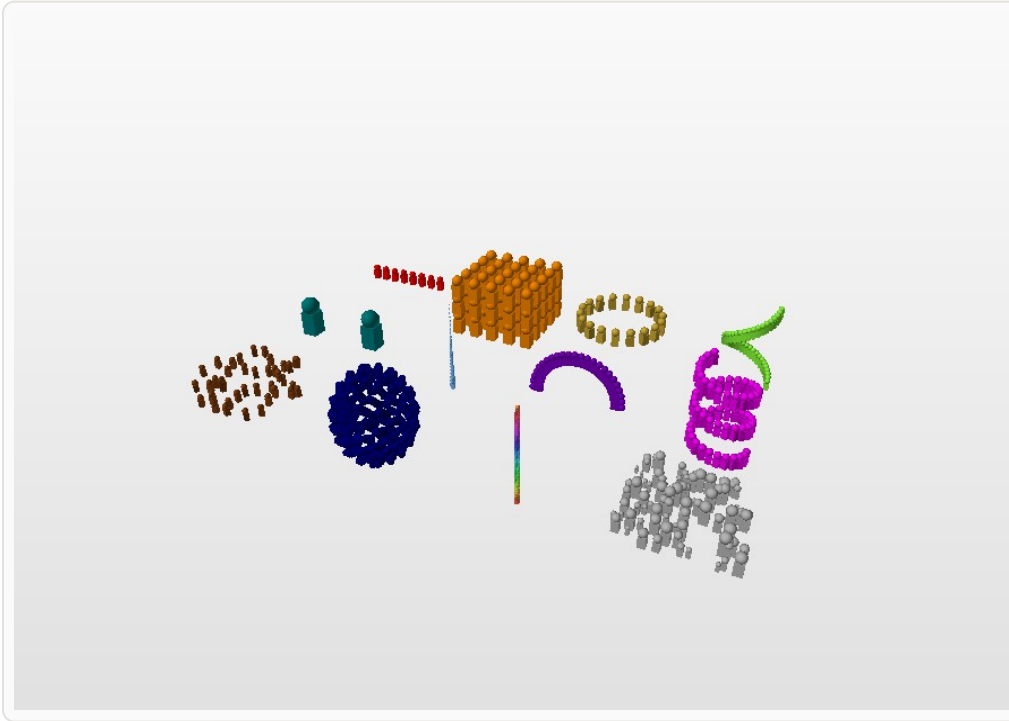
deform

Any function of yours: $f(p) \rightarrow$ new point.

All of them return a new mesh -- the original is untouched, so they chain:

```
M = add.move(add.rotateY(add.zoom(M, 0.5), 0.4), [0, 3, 0])
```

One part, many copies



```
array_linear(M, step, n)
```

```
array_grid(M, steps, counts)
```

```
array_radial(M, n, axis, P)
```

```
array_mirror(M, point, normal)
```

```
scatter(M, points, seed)
```

```
along(M, path, n)
```

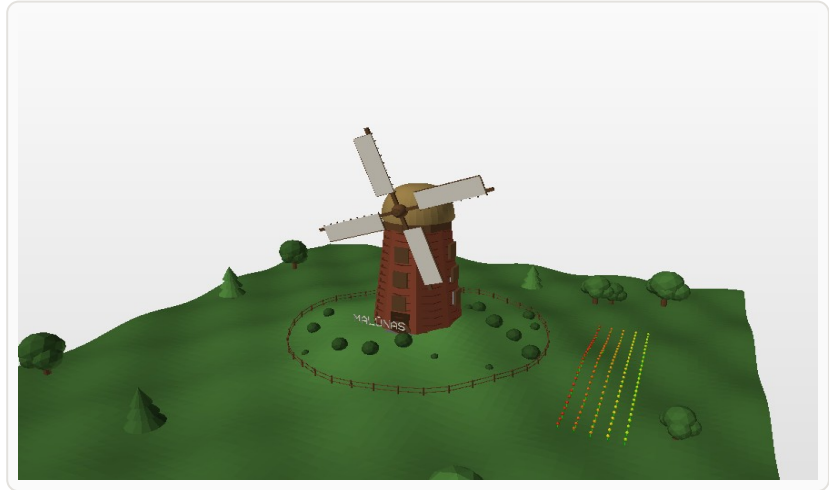
```
spiral = add.repeat(brick, 40,  
    lambda M, i: add.move(  
        add.rotateY(M, i * 0.3),  
        [0, i * 0.2, 0]))
```

NEW

Parts that models keep needing



23_locomotive.py -- wheel, beam, rotate_point, array_mirror



24_windmill.py -- extrude, arch, stairs, tree, scatter, along

beam(A, B, w, h)

A bar between two points -- it works out the orientation itself.

wheel, gear, arch, stairs

Wheels with spokes, cogs that mesh, arches, flights of steps.

roof, column, bricks, tree

Houses, temples, walls, forests.

pixels, heightmap, text

Pixel art, block terrain, labels in a built-in font (ĄČĖĮŠŲŪŽ too).

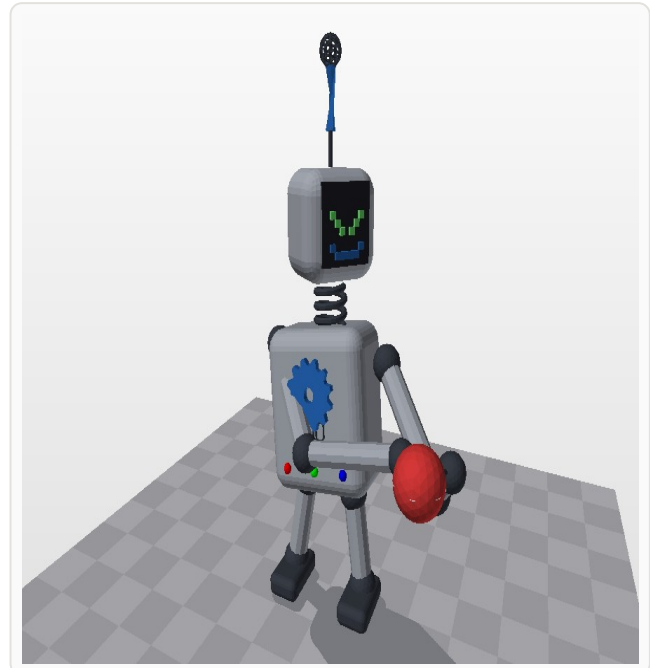
NEW

Placing: aim, scatter, along

```
# a part built around the origin, pointing up
add.push()
add.capsule([0, 0, 0], [0, 2, 0], 0.2, 12, "silver")
arm = add.pop()

# aim it from shoulder to hand, then move it
arm = add.aim(arm, add.direction(shoulder, hand))
add.mesh(add.move(arm, shoulder))

# forty trees on the hills
spots = add.random_points(40, lo, hi, seed=1,
                           height=hills)
add.mesh(add.scatter(tree, spots, seed=1,
                      scale=(0.7, 1.4)))
```



27_robot.py -- every limb aimed at a point

Build every part around the origin between `push()` and `pop()`; then aim it, turn it, and move it into place.

Colours

```
add.box(c, 1, "red")           # by name
add.box(c, 1, "#ff8800")      # hex
add.box(c, 1, [255, 136, 0])  # RGB

add.hsv(0.3, 0.7, 1.0)        # hue
add.gradient(t, "navy", "gold") # blend
add.shade("red", 0.6)          # darker

M = add.color_by(M,
    lambda p: add.hsv(p[1] / 10.0))
```

The assignment wants at least three colours -- it looks better when they mean something.

color(M, colour)

Repaints the whole mesh.

color_by(M, fn)

Colour by the position of each face -- rainbows, height maps.

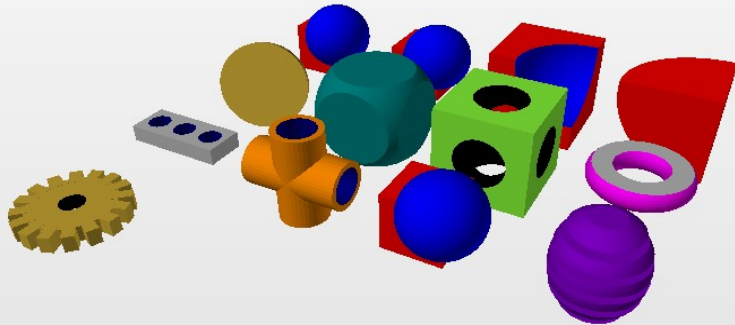
color_gradient

A blend from one colour to another along an axis.

color_random

Every face its own random colour.

Boolean operations on solids



union(A, B)

Union: everything in either solid. Hidden inner walls disappear.

intersect(A, B)

Intersection: only what is common.

difference(A, B)

Difference: A with B cut out. This is how holes and notches are made.

cut(M, point, normal)

A cut with a plane -- cheaper.

```
add.cuboid([0,0,0], [4,1,4], "brown")
plate = add.layer()
add.cylinder([0,-1,0], [0,1,0], 0.6, 32, "black")
drill = add.layer()
add.mesh(add.difference(plate, drill))
```

How the booleans work

1. Split

Every face is cut where the other solid's triangles could cross it. Neighbours come from a spatial grid, so a distant face is never cut.

2. Ask

Each piece is asked: inside, outside, or lying on the other surface? A ray counting crossings answers.

3. Keep

Union keeps what is outside; intersection what is inside; difference A's outside and B's inside, reversed.

About 600 lines of plain Python. No libraries.

A 6000-face sphere minus a box: about 0.5 s. 25 000 faces: about 2 s.

Booleans need closed solids. If the result looks odd, check both inputs with `add.check()` first.

Repairing and checking a model

clean(M)

Welds coincident vertices, drops zero-area and repeated faces, removes the inner walls where two solids touch.

heal(M)

Closes the hairline gaps where an edge passes a foreign vertex.

fix_normals(M)

Points every face the same way -- outwards, for a closed model.

triangulate(M), inside(M, p)

Every face a triangle; is a point inside the solid?

```
model = add.clean(add.layer())  
add.mesh(model)  
add.check()
```

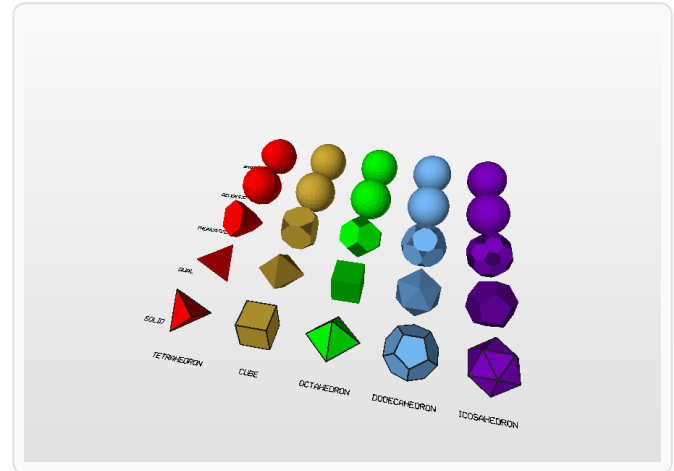
vertices	136768
OK polygons	163132
OK colours	6
OK closed surface	yes
volume	60.286

Two touching cubes each keep a wall where nobody can see it. clean() finds and removes them. examples/31_workbench.py shows every repair tool at work.

The five regular polyhedra, and a sphere of triangles

```
add.icosahedron([0, 0, 0], 2, "purple")
add.dodecahedron([5, 0, 0], 2, "gold")
P = add.polyhedron_points("icosahedron")

add.sphere([0, 5, 0], 2, 20, "sky")    # 5120 tri
add.icosphere([5, 5, 0], 2, 3, "red")  # 1280
add.quadsphere([9, 5, 0], 2, 12, "gold")
```



38_polyhedra.py -- solids, duals, truncations, domes, smooth

Centred

The average of the vertices is exactly the centre; every vertex at distance r .

The dome principle

Split every triangle in four, push the midpoints out to the sphere, repeat: 20, 80, 320, 1280 ...

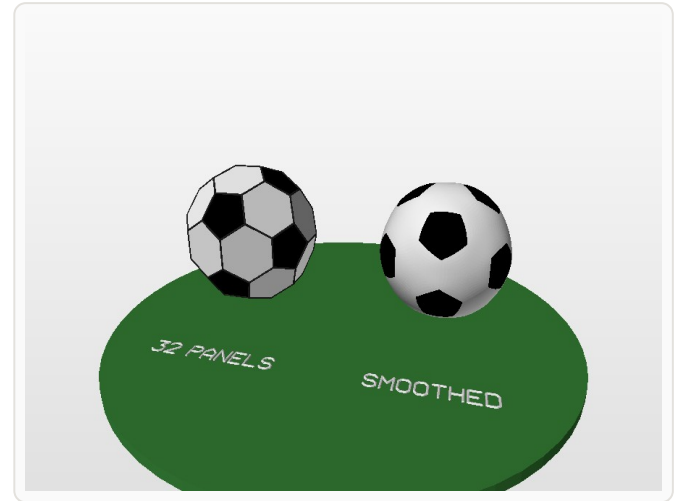
k as before

$\text{sphere}(c, r, k)$ picks the level from k : $k=10 \rightarrow 1280$, $k=20 \rightarrow 5120$ faces.

A football from the vertices of an icosahedron

```
ico = add.make(add.icosahedron, [0, 0, 0], 3)
print(add.neighbors(ico, 0))  # 5, in order
print(add.valence(ico, 0))

ball = add.truncate(ico, 1 / 3.0)  # corners off
ball = add.color_by_sides(ball,
                             {5: "black", 6:
                              "white"})
add.mesh(add.smooth(ball, 16))      # rounded
```



37_football.py

Ask the mesh

neighbors, valence,
mean_neighbor_distance,
edges, vertex_normal,
face_center, boundary_loops.

Change one corner

set_vertex(M, i, [x, None, z])
moves vertex i and keeps the
faces -- then smooth() it.

Rebuild

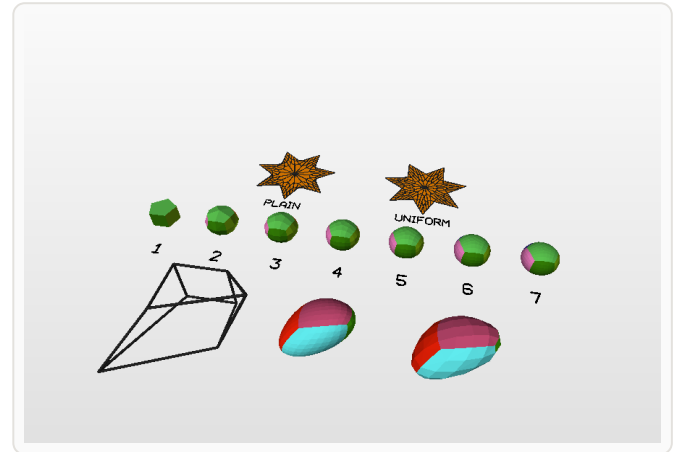
dual, truncate, refine + spherify,
inflate -- new solids from old
ones.

SMOOTH()

Smooth surfaces: generalised Catmull-Clark

```
add.box([0, 0, 0], 2, "gold")
block = add.layer()
i = add.nearest_vertex(block, [1, 1, 1])
block = add.set_vertex(block, i, [2.5, 2.5, None])

add.mesh(add.smooth(block, 8))    # 8 per edge
add.mesh(add.catmull_clark(block, 3))
```



39_smooth_shapes.py -- n = 1 ... 7 on one prism

Any n

n cells on every control edge for
n = 1, 2, 3, 4, 5 ... -- classical
subdivision only gives 2, 4, 8.

Exactly on the limit surface

Every new vertex is evaluated on
the limit surface; near odd
corners the grid is
reparameterised to stay even.

The paper

Sabaliauskas, Uniform n-grids on
Catmull-Clark limit surfaces of
arbitrary polygon meshes
(2026); a line-by-line port to
pure Python.

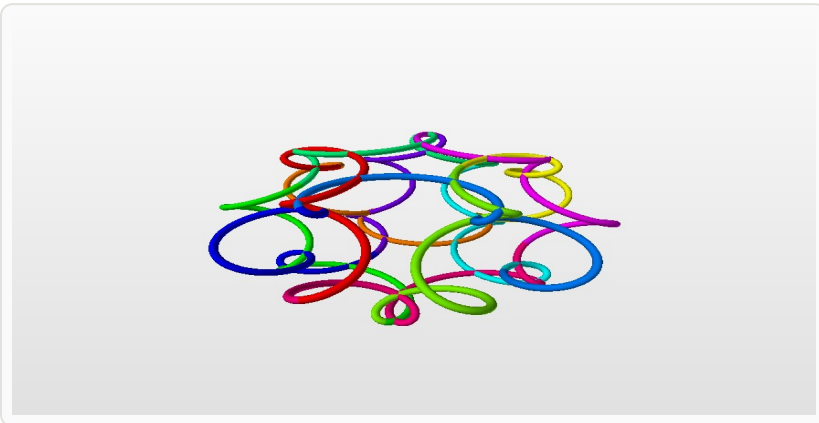
Twenty-five surfaces by name

```
print(add.surface_names())
add.surface("klein_bottle", [0, 0, 0], 4, 120)
add.surface("dini", [6, 0, 0], 4,
              color=lambda u, v: add.hsv(u / 12))
add.surface("pillow", [-6, 0, 0], 3, a=0.9)

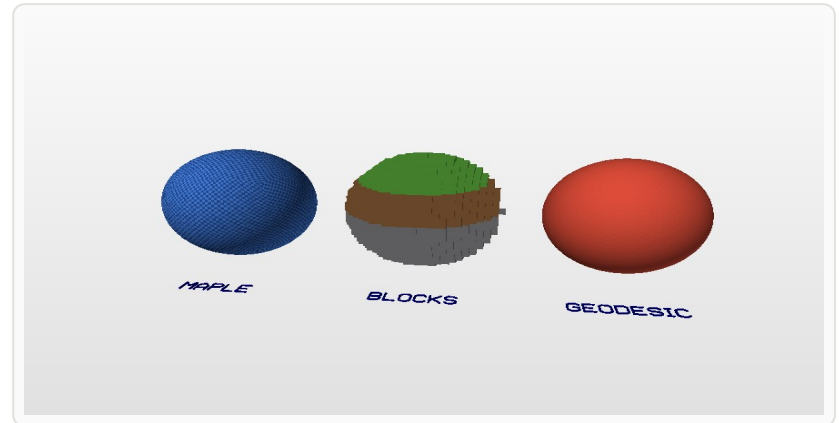
owl = add.surface_function("owl")    # bare f(u, v)
```



34_surface_zoo.py



35_knot_curve.py -- a knot from rotating circles, one curve() call

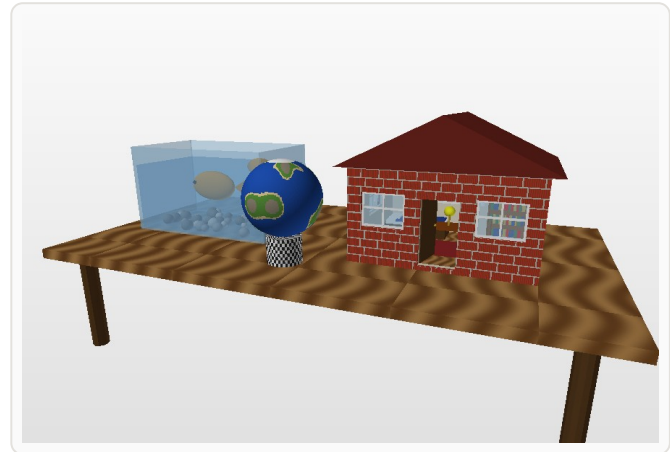


36_minecraft_sphere.py -- Maple, blocks, geodesic

Glass, pictures and the Sketchfab limits

```
glass = add.transparent("sky", 0.35)      # 0..1
wall = add.make(add.cuboid, [0, 1.5, 0], [6, 3, 0.3])
hole = add.make(add.cuboid, [0, 1.6, 0], [2, 1.2, 1])
wall = add.difference(wall, hole) # cut first,
add.mesh(add.texture(wall, "bricks.png")) # then
# the pane, in the opening
add.cuboid([0, 1.6, 0], [2, 1.2, 0.03], glass)

add.check()      # <= 50 MB, <= 50 colours
add.save("house.obj", colors=50)  # .mtl
```



45_glass_and_textures.py -- a room behind the glass

Optional

Without transparent/texture everything is exactly as before; .off files keep plain colours.

Limits

Sketchfab: 100 MB free plan, 100 materials merged beyond that. The course: 50 MB, 50 colours. `limit_colors()` and `obj_size()` help.

Pictures from code

`write_png("bricks.png", rows)`
writes a picture you computed --
no image files needed.

The castle: a model bigger than memory



the throne hall and the feast



the dragon on the treasure

no textures, every stone a polygon: an .off of about 1 GB and an .obj that 7-Zip packs into about 100 MB for Sketchfab

```
out = add.stream("castle.off", precision=4)  # parts go to disk at once,
wall_of_bricks(); out.add()                 # tidied (no overlaps), written, cleared
out.close()                                # fills in the OFF header (.mtl for .obj)
```

The castle on Sketchfab, to turn round in 3D: skfb.ly/pOnRS

Saving and sharing a model

```
add.save("model.off")    # the course format
add.save("model.obj")    # + .mtl for colours
add.save("model.ply")
add.save("model.stl")    # for 3D printing
```

The MeshLab conversion step is gone
-- add.py writes OBJ and MTL itself.

Sketchfab

Put .obj and .mtl in one archive and upload -- anyone can turn the model in a browser, like the castle:

skfb.ly/pOnRS

3D printing

clean(..., normals=True), then save("m.stl"). The surface must be closed.

Preview without MeshLab

python3 tools/preview.py model.off writes a PNG. No libraries either.

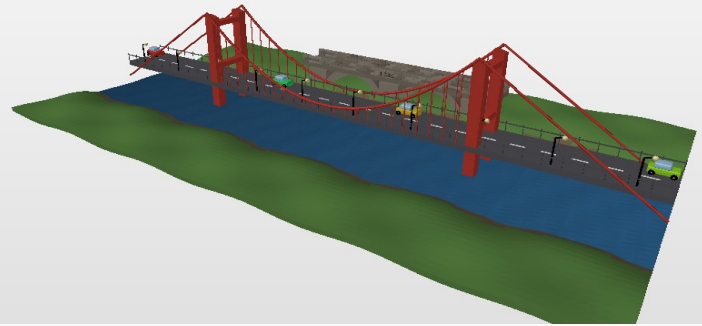
Loading back

load("letter.off") returns a mesh you can move and paint.

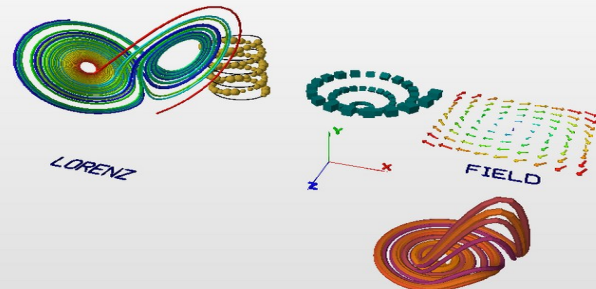
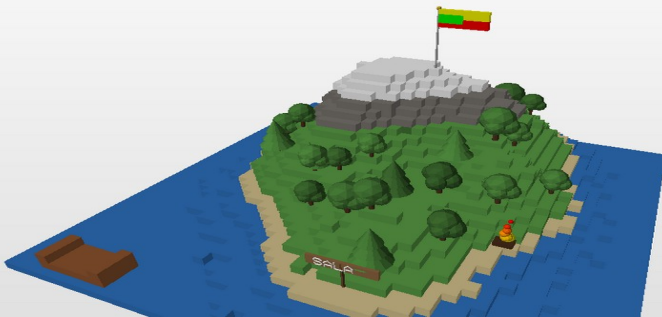
Complete models



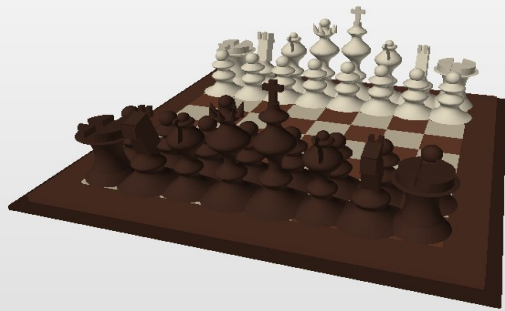
25_temple.py -- columns on a circle, a chequered floor



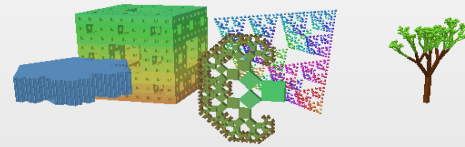
29_bridge.py -- beams, cables, hangers, cars along the road



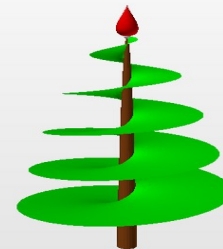
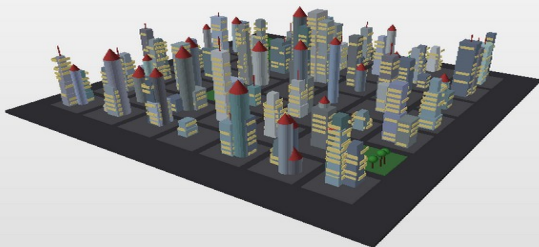
Larger studies



18_chess_set.py -- lathe, layers, booleans



17_fractals.py -- Menger sponge, Sierpinski, a tree



From 1.2 to 2.0

add.py 1.2	add.py 2.0	what it is
<code>cube2(c, e, b, RGB)</code>	frame	a hollow cube of bars
<code>cylinder2(A, B, r, k, RGB)</code>	tube	a cylinder with no lids
<code>cylinder3(A, B, r, k, RGB)</code>	cup	closed at one end
<code>cone2(A, B, r, k, RGB)</code>	cone_open	the slanted wall only
<code>rectangle3D(c, e, RGB)</code>	cuboid	a rectangular block
<code>newface(A, RGB)</code>	polygon	one flat face
<code>spin3D(A, B, S, ...)</code>	revolve	a lathe
<code>off(file)</code>	save(file)	.off, .obj, .ply or .stl

All the old names still work. Twelve models written for 1.2 live in tests/legacy/, and the tests check that they produce exactly as many faces as before (only sphere changed: it is geodesic now; quadisphere is the old one). examples/32_old_names.py is a whole model in the 1.2 vocabulary.

Where to find everything

add.py and add.hpp, the examples, the tests and the documentation

github.com/.../add3d

Documentation in English and Lithuanian

...github.io/add3d

Questions

akataxis@gmail.com

Thank you.

